

Module – String Immutability & Intern Pool

String Basics

Key Points

- `String` in Java is a **class**, not a primitive.
- It represents a sequence of characters (`char[]`).
- Strings are **immutable** → once created, cannot be changed.
- Stored in the **String Constant Pool (SCP)** inside the heap.

Code Snippet

```
String s1 = "Learn";  
String s2 = "Learn";  
System.out.println(s1 == s2); // true
```

 **Exercise:** `Exercise_01_LiteralPooling.java`

Predict whether two strings created using literals or `new` keyword refer to same object.

Immutability Explained

Key Points

- Once a string object is created, its content cannot change.
- Operations like `concat()`, `+`, or `replace()` return a **new object**.
- The original reference still points to the old value.

Code Snippet

```
String s = "Java";  
s.concat("21");  
System.out.println(s); // "Java"
```

 **Exercise:** `Exercise_02_StringConcatImmutability.java`

Trace memory references before and after `concat()` or `+` operations.

Interning and the String Pool

Key Points

- The **String Pool** holds one copy of each unique literal.
- Calling `intern()` returns the pooled version of a string.
- Saves memory and improves performance.

Code Snippet

```
String s1 = new String("Hello");
String s2 = "Hello";
System.out.println(s1 == s2);           // false
System.out.println(s1.intern() == s2);  // true
```

 **Exercise:** Exercise_03_InterningBehavior.java
Compare strings created using `new` and interned literals.

When to Use Interning

Key Points

- Use for **frequently repeated strings** (e.g., IDs, constants).
- Automatically applied to literals at class load time.
- Manual `intern()` can reduce duplicate heap strings in large apps.
- Overuse may bloat the pool → use carefully.

 **Exercise:** `Exercise_04_InternPerformance.java`

Create thousands of repeated strings with and without `intern()`; measure memory usage.

String Creation Techniques

Key Points

- Literal ("text") → stored in pool automatically.
- `new String("text")` → creates new heap object.
- `intern()` → adds to pool if not already there.

Code Snippet

```
String a = "Hi";  
String b = new String("Hi");  
String c = b.intern();  
System.out.println(a == b); // false  
System.out.println(a == c); // true
```

 **Exercise:** Exercise_05_StringCreation.java
Analyze which references point to same pool instance.

Common Pitfalls

Key Points

- Using `==` compares references, not content.
- Always use `.equals()` to compare string values.
- `intern()` doesn't copy — it reuses or adds to pool.

Code Snippet

```
String x = "Test";  
String y = new String("Test");  
System.out.println(x == y);      // false  
System.out.println(x.equals(y));  // true
```

 **Exercise:** `Exercise_06_EqualsVsDoubleEquals.java`
Distinguish between reference and content equality.

Memory Diagram Recap

Visual

Heap:
└─ new String("Java") → separate object
String Pool:
└─ "Java" → shared literal

 **Exercise:** Exercise_07_MemoryTracing.java

Draw diagrams showing which objects are reused or duplicated.

Key Takeaways

- Strings are **immutable** → safer, thread-safe, memory-efficient.
- Literals are automatically **pooled**.
- `intern()` links heap strings back to the pool.
- Use `.equals()` for value comparison.
- Reassign after concatenation if you expect change.

Module – String Concatenation Rules & Compiler Optimizations

Concatenation Basics

Key Points

- String concatenation joins multiple strings into one.
- Performed using `+` operator or `concat()` method.
- Compiler converts `+` operations on literals to a single string at **compile time**.
- When variables are involved, concatenation happens at **runtime**.

Code Snippet

```
String s1 = "Hello " + "World";  
System.out.println(s1); // "Hello World"
```

 **Exercise:** `Exercise_01_CompileTimeConcat.java`
Check bytecode using `javap -c` to verify constant folding.

Compile-Time Optimization (Constant Folding)

Key Points

- When all operands are constants/literals, compiler performs **constant folding**.
- Result is stored directly in **String Constant Pool (SCP)**.
- No new StringBuilder or runtime concatenation occurs.

Code Snippet

```
String s1 = "Java";  
String s2 = "21";  
String s3 = "Java" + "21";  
System.out.println(s3 == "Java21"); // true
```

 **Exercise:** Exercise_02_ConstantFolding.java

Try mixing literals and variables to observe SCP behavior.

Runtime Concatenation (StringBuilder Usage)

Key Points

- When variables are involved, compiler uses **StringBuilder** for efficiency.
- Code like `s1 + s2 + s3` becomes chained `append()` calls internally.
- Each `+` does **not** create a new intermediate string.

Code Snippet

```
String s1 = "Hello";  
String s2 = "World";  
String s3 = s1 + s2;  
System.out.println(s3); // "HelloWorld"
```

 **Exercise:** Exercise_03_RuntimeConcat.java

Use `javap -c` to see **StringBuilder** usage behind `+`.

Mixing Literals and Variables

Key Points

- If even one operand is variable, entire expression is evaluated at **runtime**.
- Resulting string is created on the **heap**, not in SCP.
- You can use `intern()` to move it to the pool manually.

Code Snippet

```
String s1 = "Java";  
String s2 = s1 + "21";  
System.out.println(s2 == "Java21"); // false  
System.out.println(s2.intern() == "Java21"); // true
```

 **Exercise:** Exercise_04_MixedConcat.java

Predict which concatenations produce pooled vs heap strings.

Performance Considerations

Key Points

- Using `+` inside loops repeatedly creates many intermediate objects.
- Prefer **StringBuilder** or **StringBuffer** for performance in loops.
- Compiler optimizes `+` only within a single statement, not across iterations.

Code Snippet

```
String result = "";
for (int i = 0; i < 5; i++) {
    result = result + i;
}
System.out.println(result);
```

 **Exercise:** Exercise_05_ConcatPerformance.java

Rewrite loop using `StringBuilder` and compare execution time.

Using StringBuilder Explicitly

Key Points

- `StringBuilder` is mutable — best for repeated concatenations.
- Use `StringBuffer` if thread safety is needed.
- Convert back to `String` only once using `toString()`.

Code Snippet

```
StringBuilder sb = new StringBuilder("Learn");  
sb.append(" ").append("Java");  
System.out.println(sb.toString());
```

 **Exercise:** `Exercise_06_StringBuilderUsage.java`
Compare memory and speed between `+` vs `StringBuilder`.

Special Case: Compile-Time Evaluation with Final Variables

Key Points

- If a variable is `final` and initialized with a literal, compiler treats it as a constant.
- Concatenation using only `final` constants may also be optimized at compile time.

Code Snippet

```
final String a = "Hello";  
final String b = "World";  
String c = a + b;  
System.out.println(c == "HelloWorld"); // true
```

 **Exercise:** `Exercise_07_FinalConstConcat.java`
Modify constants to non-final and check equality.

Key Takeaways

- `+` is compile-time optimized when operands are literals/constants.
- Otherwise, compiler uses **StringBuilder** at runtime.
- Use `StringBuilder` or `StringBuffer` for performance-critical loops.
- `intern()` links heap strings back to pool for memory reuse.
- Understand difference between **SCP** (compile-time) vs **Heap** (runtime).

Module – Methods in String and StringBuilder

String Class Overview

Key Points

- `String` class is **immutable** — every modification creates a new object.
- Provides rich methods for comparison, searching, and transformation.
- Stored in the **java.lang** package.
- Commonly used for text manipulation and user inputs.

Code Snippet

```
String s = "Java Programming";  
System.out.println(s.length()); // 16  
System.out.println(s.toUpperCase()); // "JAVA PROGRAMMING"
```

 **Exercise:** `Exercise_01_StringBasics.java`

Test `length()`, `toUpperCase()`, and `toLowerCase()` on same string and check object references.

Searching & Comparing Strings

Key Points

• Methods for searching and comparison:

- `charAt(int index)`
- `indexOf(String str) / lastIndexOf(String str)`
- `contains(CharSequence s)`
- `equals() / equalsIgnoreCase() / compareTo()`

Code Snippet

```
String s = "Developer";
System.out.println(s.charAt(2));           // 'v'
System.out.println(s.indexOf("el"));       // 2
System.out.println(s.contains("Dev"));     // true
System.out.println(s.equals("developer")); // false
```

 **Exercise:** `Exercise_02_SearchCompare.java`

Find positions of substrings and compare text ignoring case.

Substring and Replacement Methods

Key Points

- `substring(int begin, int end)` → extracts part of string.
- `replace(char old, char new)` → replaces all occurrences.
- `replaceAll(regex, replacement)` → uses regex pattern.
- All methods return **new String objects**.

Code Snippet

```
String s = "Java 21 Developer";
System.out.println(s.substring(0, 4));      // "Java"
System.out.println(s.replace('a', '@'));    // "J@v@ 21 Developer"
System.out.println(s.replaceAll("\\d", "XX")); // "Java XX Developer"
```

 **Exercise:** Exercise_03_SubReplace.java

Extract words and replace digits or vowels using different methods.

Trimming, Splitting, and Joining

Key Points

- `trim()` → removes leading and trailing spaces.
- `strip()`, `stripLeading()`, `stripTrailing()` → Unicode-aware (Java 11+).
- `split(String regex)` → splits string into an array.
- `join(CharSequence delimiter, elements...)` → concatenates with delimiter.

Code Snippet

```
String data = "  Alpha, Beta , Gamma ";
System.out.println(data.trim()); // "Alpha, Beta , Gamma"
String[] parts = data.split(",");
System.out.println(String.join(" | ", parts)); // "  Alpha | Beta |
Gamma "
```

 **Exercise:** Exercise_04_SplitJoin.java
Split a CSV line and rejoin using different delimiters.

StringBuilder Overview

Key Points

- `StringBuilder` is **mutable** → contents can be modified in place.
- Ideal for loops or repeated concatenations.
- Not thread-safe (use `StringBuffer` if needed).
- Located in **java.lang** package.

Code Snippet

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" ").append("World");  
System.out.println(sb); // "Hello World"
```

 **Exercise:** `Exercise_05_StringBuilderIntro.java`

Use `append()` in loop and check performance difference from `String`.

Common StringBuilder Methods

Key Points

- `append()` → adds text to end.
- `insert(int index, String str)` → adds text in middle.
- `delete(int start, int end)` → removes characters.
- `reverse()` → reverses sequence.
- `replace(int start, int end, String str)` → replaces range.

Code Snippet

```
StringBuilder sb = new StringBuilder("World");  
sb.insert(0, "Hello ");  
sb.delete(5, 6);  
sb.reverse();  
System.out.println(sb); // "dlroW olleH"
```

 **Exercise:** `Exercise_06_BuilderMethods.java`

Perform chained modifications and observe how object reference remains same.

Converting Between String and StringBuilder

Key Points

- Convert `String` → `StringBuilder` using constructor.
- Convert `StringBuilder` → `String` using `toString()`.
- Useful when switching between immutable and mutable operations.

Code Snippet

```
String s = "Java";
StringBuilder sb = new StringBuilder(s);
sb.append("21");
String result = sb.toString();
System.out.println(result); // "Java21"
```

 **Exercise:** `Exercise_07_Conversion.java`

Modify a `String` through `StringBuilder` and verify object references.

Performance Comparison

Key Points

- `String` is fine for few concatenations.
- `StringBuilder` is faster for repeated appends or modifications.
- Use `StringBuffer` for multi-threaded environments.
- Measure time using `System.nanoTime()` or JMH benchmarks.

Code Snippet

```
String s = "";
for (int i = 0; i < 1000; i++) s += i;
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i);
```

 **Exercise:** `Exercise_08_Performance.java`

Compare runtime and memory consumption of `String` vs `StringBuilder`.

Key Takeaways

- `String` → immutable; best for fixed text and literals.
- `StringBuilder` → mutable; ideal for dynamic and repeated updates.
- Many `String` methods return **new objects** — reassignment required.
- Choose `StringBuilder` for performance-critical concatenations.
- Convert between both as needed using constructors and `toString()`.

Module – Equality, compareTo & Substring Boundaries

Reference vs Content Equality

Key Points

- `==` compares **object references**, not the content.
- `.equals()` compares **values/content** inside the objects.
- String literals may refer to same object in the **String Pool**, but `new String()` creates a new heap object.

Code Snippet

```
String a = "Java";  
String b = "Java";  
String c = new String("Java");  
System.out.println(a == b);           // true (same SCP object)  
System.out.println(a == c);           // false (heap object)  
System.out.println(a.equals(c));      // true (same content)
```

 **Exercise:** `Exercise_01_EqualityBasics.java`

Predict outcomes of `==` and `.equals()` for literals and heap strings.

Understanding .equals() Method

Key Points

- `equals()` in `String` is **overridden** from `Object`.
- It checks character sequences for equality.
- Case-sensitive: `"Java".equals("java") → false`.
- Use `equalsIgnoreCase()` for case-insensitive comparison.

Code Snippet

```
String x = "Hello";  
String y = "hello";  
System.out.println(x.equals(y));           // false  
System.out.println(x.equalsIgnoreCase(y)); // true
```

 **Exercise:** `Exercise_02_EqualsVsIgnoreCase.java`

Compare various case combinations and see where `equalsIgnoreCase` helps.

Using compareTo() Method

Key Points

- `compareTo()` compares strings **lexicographically** (dictionary order).
- Returns:
 - 0 if both are equal,
 - < 0 if current string is smaller,
 - > 0 if current string is larger.
 - Based on Unicode value of characters.

Code Snippet

```
System.out.println("Apple".compareTo("Banana")); // negative
System.out.println("Banana".compareTo("Apple")); // positive
System.out.println("Cat".compareTo("Cat"));      // 0
```

 **Exercise:** `Exercise_03_CompareToDemo.java`

Sort an array of strings using `compareTo()` manually and verify with `Arrays.sort()`.

compareToIgnoreCase()

Key Points

- `compareToIgnoreCase()` compares two strings ignoring case.
- Useful in sorting or searching where case doesn't matter.

Code Snippet

```
System.out.println("Java".compareToIgnoreCase("java")); // 0
```

 **Exercise:** `Exercise_04_CompareIgnoreCase.java`

Sort a list of mixed-case words and observe results with both methods.

Substring Boundaries Basics

Key Points

- `substring(beginIndex, endIndex)` extracts a part of a string.
- `beginIndex` → **inclusive**, `endIndex` → **exclusive**.
- If `endIndex > length()` or `beginIndex > endIndex`, `StringIndexOutOfBoundsException` occurs.

Code Snippet

```
String s = "Developer";
System.out.println(s.substring(0, 3)); // "Dev"
System.out.println(s.substring(3));    // "eloper"
```

 **Exercise:** `Exercise_05_SubstringBoundaries.java`

Try extracting last characters using `length()` to avoid exceptions.

Common Substring Errors

Key Points

- Out-of-bound indices cause runtime exceptions.
- Always validate indices before calling `substring()`.
- `substring()` returns a **new String object** (immutability maintained).

Code Snippet

```
String s = "Java";  
System.out.println(s.substring(4)); //  Exception (end > length)
```

 **Exercise:** `Exercise_06_SubstringErrors.java`
Catch exceptions and print custom error messages.

Combining Substring and Equality

Key Points

- You can compare substrings directly using `.equals()` or `compareTo()`.
- Substring creates a new object → `==` will be false even if text matches.

Code Snippet

```
String s1 = "Learning Java";  
String s2 = s1.substring(9); // "Java"  
System.out.println(s2 == "Java");           // false  
System.out.println(s2.equals("Java"));       // true
```

 **Exercise:** `Exercise_07_SubstringEquality.java`

Extract parts of strings and check equality with literals using both operators.

Edge Cases & Practical Tips

Key Points

- Use `.equals()` or `.equalsIgnoreCase()` for safe string comparison.
- Use `compareTo()` only for sorting or ordering logic.
- When using `substring()`, always check string length to avoid exceptions.
- Prefer helper methods like `startsWith()`, `endsWith()` when applicable.

Code Snippet

```
String s = "Java21";
System.out.println(s.startsWith("Java")); // true
System.out.println(s.endsWith("21"));     // true
```

 **Exercise:** `Exercise_08_PracticalTips.java`

Rewrite conditions using `startsWith()` / `endsWith()` instead of manual `substring` checks.

Key Takeaways

- `==` → compares references; `.equals()` → compares content.
- `compareTo()` → defines natural ordering based on Unicode.
- Use `compareToIgnoreCase()` for case-insensitive sorting.
- `substring(begin, end)` → end is exclusive; out-of-bounds cause errors.
- Substrings are **new objects** → never rely on `==`.

Module – String Formatting (String.format & Formatter)

Why String Formatting?

Key Points

- Used to create **well-structured, readable output**.
- Supports placeholders for numbers, strings, dates, and more.
- Reduces manual concatenation and improves internationalization.
- Based on **C-style format specifiers** like %d, %s, %f, etc.

Code Snippet

```
int score = 95;
String name = "Nitya";
System.out.println(String.format("%s scored %d marks", name, score));
```

 **Exercise:** Exercise_01_BasicFormatting.java

Create formatted sentences for name, score, and rank using placeholders.

Common Format Specifiers

Key Points

Type	Specifier	Example	Output
Integer	%d	<code>String.format("%d", 42)</code>	42
Floating point	%f	<code>String.format("%.2f", 12.3456)</code>	12.35
String	%s	<code>String.format("%s", "Java")</code>	Java
Character	%c	<code>String.format("%c", 'A')</code>	A
Boolean	%b	<code>String.format("%b", true)</code>	true

 **Exercise:** `Exercise_02_Specifiers.java`
Practice all specifiers and predict the formatted output.

Controlling Width, Alignment & Padding

Key Points

- Specify **field width** and **alignment**:

- `%5d` → pad with spaces (right-aligned).
- `%-5d` → left-aligned.
 - Pad with zeros using `0`:
- `%05d` → pads with zeros (e.g., `00042`).
 - Useful for tabular or columnar output.

Code Snippet

```
System.out.println(String.format("%5d", 42));    // "    42"
System.out.println(String.format("%-5d", 42));   // "42    "
System.out.println(String.format("%05d", 42));   // "00042"
```

 **Exercise:** `Exercise_03_AlignmentPadding.java`

Format employee IDs and names in neatly aligned columns.

Formatting Floating-Point Numbers

Key Points

- Precision: `%.2f` → two digits after decimal.
- Field width + precision: `%8.2f` → 8 chars wide, 2 decimals.
- Use for currency, percentages, averages.

Code Snippet

```
double price = 1234.56789;  
System.out.println(String.format("₹%.2f", price)); // ₹1234.57  
System.out.println(String.format("%8.2f", price)); // " 1234.57"
```

 **Exercise:** `Exercise_04_FloatPrecision.java`

Format prices and percentages with various precisions.

Formatting Strings and Characters

Key Points

- `%s` — formats any object via `toString()`.
- Supports width and alignment like numbers.
- Null-safe — displays “null” if value is null.

Code Snippet

```
String lang = null;  
System.out.println(String.format("Language: %10s", lang)); // "Language:  
null"
```

 **Exercise:** `Exercise_05_StringWidth.java`

Right-align and left-align names and departments.

Using Formatter Class (java.util.Formatter)

Key Points

- `Formatter` provides **stream-based** formatted output.
- Can write to `System.out`, files, or `StringBuilder`.
- Supports same format specifiers as `String.format()`.

Code Snippet

```
import java.util.*;

Formatter f = new Formatter();
f.format("Name: %s, Marks: %d", "Ravi", 90);
System.out.println(f); // Name: Ravi, Marks: 90
f.close();
```

 **Exercise:** `Exercise_06_FormatterIntro.java`

Use `Formatter` to write formatted text to console and to a `StringBuilder`.

Formatting to File or StringBuilder

Key Points

- `Formatter` can direct output to various destinations:
 - Console → `new Formatter(System.out)`
 - File → `new Formatter("output.txt")`
 - `StringBuilder` → `new Formatter(sb)`
 - Ideal for reports or logs.

Code Snippet

```
import java.io.*;
import java.util.*;

try (Formatter f = new Formatter("report.txt")) {
    f.format("Employee: %s | Salary: %.2f", "Anita", 98765.432);
}
```

 **Exercise:** `Exercise_07_FormatterFile.java`
Generate a formatted report in a file using `Formatter`.

Locale-Specific Formatting

Key Points

- Use `Locale` to format numbers or currency regionally.
- `String.format(Locale, format, args)` supports `il8n`.
- `Locale` affects decimal separator, currency symbol, etc.

Code Snippet

```
import java.util.*;

double amt = 12345.67;
System.out.println(String.format(Locale.US, "%.2f", amt));    // 12,345.67
System.out.println(String.format(Locale.FRANCE, "%.2f", amt)); // 12
345,67
```

 **Exercise:** `Exercise_08_LocaleFormatting.java`
Format the same number for US, France, and India.

Combining Everything

Key Points

- You can mix multiple specifiers in one format string.
- Use indexes to reorder arguments: %2\$s refers to 2nd arg.
- Helpful when localizing messages.

Code Snippet

```
String result = String.format("Name: %2$s, ID: %1$d", 101, "Rohan");  
System.out.println(result); // "Name: Rohan, ID: 101"
```

 **Exercise:** Exercise_09_MultiArgFormatting.java

Create complex messages mixing text, numbers, and floating-point values.

Key Takeaways

- `String.format()` is concise and powerful for textual output.
- `Formatter` class adds stream/file flexibility.
- Use width, precision, and alignment for professional-looking data.
- Locale-aware formatting supports global applications.
- Index-based specifiers simplify translation and dynamic templates.

Module – Text Blocks (" " ") – Whitespace Control & Embedded Quotes

What Are Text Blocks?

Key Points

- Text Blocks are **multi-line string literals** introduced in Java 15.
- Use triple double-quotes (" " ") instead of single quotes.
- Make multi-line strings easier to read and maintain.
- Automatically handle line breaks (\n) and quote escaping.

Code Snippet

```
String html = """
    <html>
        <body>
            <h1>Hello, World!</h1>
        </body>
    </html>
    """;
System.out.println(html);
```

 **Exercise:** Exercise_01_TextBlockIntro.java

Convert a multi-line HTML string from traditional format to a Text Block.

Syntax & Rules

Key Points

- Starts and ends with triple double quotes """.
- First line break after the opening """ is **not included** in the value.
- Closing """ must appear on its own line.
- Indentation of closing quotes determines how common whitespace is stripped.

Code Snippet

```
String poem = ""  
    Roses are red,  
    Violets are blue,  
    Code is elegant,  
    When written by you.  
    """;
```

 **Exercise:** Exercise_02_SyntaxRules.java

Experiment by moving the closing """ left or right and print to observe indentation trimming.

Whitespace Control (Indentation & Alignment)

Key Points

- Text Blocks automatically remove common leading whitespace.
- You can align the closing delimiter `"""` to control indentation.
- If a line has less indentation than others, Java keeps those extra spaces.
- To preserve all indentation exactly, align closing quotes to column 0.

Code Snippet

```
String json = """
    {
        "id": 101,
        "name": "Rohan"
    }
    """;
System.out.println(json);
```

 **Exercise:** Exercise_03_WhitespaceControl.java

Adjust indentation of text blocks and observe how Java normalizes whitespace

Embedded Quotes and Escapes

Key Points

- You can directly include **double quotes** (") inside text blocks — no escaping needed.
- Only the sequence of **three quotes** (""") needs escaping (\ """).
- Newlines are implicit; you don't need \n.

Code Snippet

```
String quote = """"
    She said, "Java is elegant."
    He replied, "Indeed!"
    """;
System.out.println(quote);
```

 **Exercise:** Exercise_04_EmbeddedQuotes.java

Write a text block with embedded dialogue lines without any escape characters.

Including Escapes & Special Characters

Key Points

- You can still use escape sequences if needed:
 - `\t, \n, \\", \"`
 - Text Blocks preserve line breaks but ignore trailing spaces.
 - For trailing whitespace, use `\s` escape.

Code Snippet

```
String data = ""  
    Path: C:\\Program Files\\Java\\  
    Tabs:\\tused here  
    Trailing space->\\s  
    "";
```

 **Exercise:** `Exercise_05_SpecialChars.java`

Print text with tabs, slashes, and trailing spaces to observe formatting.

Combining Text Blocks with String Methods

Key Points

- Text Blocks are still `String` objects — you can call methods like `stripIndent()`, `formatted()`, etc.
- `stripIndent()` removes incidental indentation uniformly.
- `formatted()` allows inserting values like `String.format()`.

Code Snippet

```
String template = """
    Name: %s
    Age: %d
    City: %s
    """.formatted("Nitya", 14, "Bangalore");

System.out.println(template);
```

 **Exercise:** `Exercise_06_TextBlockTemplate.java`

Create a JSON-like text block and fill placeholders using `.formatted()`.

Multi-line JSON, XML, and SQL Examples

Key Points

- Text Blocks make JSON, XML, SQL, or HTML much more readable.
- No need for manual escaping or concatenation.
- Makes code self-documenting and easier to debug.

Code Snippet

```
String sql = ""  
    SELECT id, name, salary  
    FROM employees  
    WHERE department = "Sales"  
    ORDER BY salary DESC;  
    "";
```

 **Exercise:** Exercise_07_SQLandJSON.java

Write SQL and JSON text blocks and use them in mock methods.

Text Block vs Regular String Comparison

Key Points

Feature	Regular String	Text Block
Multi-line	✗ Needs \n	✓ Native support
Quotes inside	Must escape \"	Allowed directly
Readability	Poor for long text	Excellent
Whitespace control	Manual	Automatic
Use case	Short text	Large structured text

 **Exercise:** `Exercise_08_CompareOldVsNew.java`

Rewrite an HTML or SQL snippet from old format to Text Block format.

Key Takeaways

- Text Blocks simplify multi-line string creation and readability.
- Automatic **whitespace normalization** based on closing `"""`.
- Support **embedded quotes** and line breaks naturally.
- Work seamlessly with `.formatted()` and other string methods.
- Ideal for **JSON, SQL, XML, HTML, and documentation** in code.

Module – Regular Expressions Basics with Pattern and Matcher

What Is a Regular Expression?

Key Points

- A **regular expression (regex)** is a sequence of characters defining a search pattern.
- Used for matching, validating, and extracting text.
- Supported by `java.util.regex` package.
- Core classes:
 - `Pattern` → defines the compiled regex.
 - `Matcher` → performs the actual search or match.

Code Snippet

```
import java.util.regex.*;

Pattern p = Pattern.compile("Java");
Matcher m = p.matcher("I love Java programming!");
System.out.println(m.find()); // true
```

 **Exercise:** `Exercise_01_BasicMatch.java`

Write a regex that finds a given word in a sentence.

Pattern and Matcher Workflow

Key Points

- 1 ☐ **Compile** regex using `Pattern.compile(regex)`
- 2 ☐ **Create Matcher** using `pattern.matcher(input)`
- 3 ☐ **Use methods** like `find()`, `matches()`, `group()`, etc.

Code Snippet

```
Pattern pattern = Pattern.compile("\\d+");
Matcher matcher = pattern.matcher("There are 25 students.");
if (matcher.find()) {
    System.out.println("Found number: " + matcher.group());
}
```

 **Exercise:** `Exercise_02_PatternMatcherFlow.java`
Detect all numbers in a sentence using `find()` in a loop.

Common Regex Symbols

Key Points

Symbol	Meaning	Example	Matches
.	Any character	a.b	ab, a-b, acb
\d	Digit	\d\d	12
\w	Word character (a-z, A-Z, 0-9, _)	\w+	hello123
\s	Whitespace	\s	space, tab
^	Start of line	^Java	"Java is fun"
\$	End of line	end\$	"the end"



Exercise:

Exercise_03_RegexSymbols.java

Experiment with ., \d, and \w to extract parts of text.

Quantifiers and Character Classes

Key Points

- Quantifiers define **how many** times a pattern repeats.
- Character classes define **sets of allowed characters**.

Symbol	Meaning	Example	Matches
*	0 or more	a*	"", a, aa
+	1 or more	a+	a, aa
?	0 or 1	colou?r	color, colour
{n}	Exactly n	\d{3}	123
[abc]	One of a, b, c	[ch]at	cat, hat
[^abc]	Not a, b, or c	[^0-9]	non-digits

Code Snippet

```
Pattern p = Pattern.compile("\\d{2,4}");  
Matcher m = p.matcher("2025 is here");  
System.out.println(m.find()); // true
```

 **Exercise:** Exercise_04_Quantifiers.java
Match varying lengths of digits, words, or dates.

Matching vs. Finding

Key Points

- `matches()` → checks if the **entire input** matches pattern.
- `find()` → looks for **next occurrence** of pattern within input.
- `find()` can be used in loops to find multiple matches.

Code Snippet

```
Pattern p = Pattern.compile("\\d+");
Matcher m = p.matcher("ID: 101, Code: 202, Pin: 303");
while (m.find()) {
    System.out.println(m.group());
}
```

 **Exercise:** Exercise_05_FindVsMatches.java

Use both methods on same input and compare outputs.

Extracting Groups

Key Points

- Groups capture sub-patterns using parentheses ().
- Use `group(n)` to access captured values.
- Group 0 → entire match.

Code Snippet

```
Pattern p = Pattern.compile("(\\w+)@(\\w+\\.\\w+)");
Matcher m = p.matcher("Contact: support@learnato.com");
if (m.find()) {
    System.out.println("User: " + m.group(1));
    System.out.println("Domain: " + m.group(2));
}
```

 **Exercise:** Exercise_06_CapturingGroups.java
Extract usernames and domains from email addresses.

Useful Methods in Matcher

Key Points

Method	Description
<code>find()</code>	Finds next match.
<code>group()</code>	Returns matched substring.
<code>groupCount()</code>	Returns number of capturing groups.
<code>start()</code> / <code>end()</code>	Returns start/end indices of match.
<code>reset()</code>	Resets matcher to beginning.

Code Snippet

```
Pattern p = Pattern.compile("\\d+");
Matcher m = p.matcher("A12B345C6");
while (m.find()) {
    System.out.println("Found " + m.group() + " at " + m.start());
}
```

 **Exercise:** `Exercise_07_MatcherMethods.java`

Print positions of all numeric matches in a given string.

Flags and Case Insensitivity

Key Points

• Flags modify pattern behavior:

- `Pattern.CASE_INSENSITIVE` → ignore case.
- `Pattern.MULTILINE` → `^` and `$` work per line.
- `Pattern.DOTALL` → `.` matches newlines.
 - Set using `Pattern.compile(regex, flag)`.

Code Snippet

```
Pattern p = Pattern.compile("java", Pattern.CASE_INSENSITIVE);
Matcher m = p.matcher("I love JAVA and JaVaScript!");
while (m.find()) System.out.println(m.group());
```

 **Exercise:** `Exercise_08_RegexFlags.java`

Test `CASE_INSENSITIVE` and `DOTALL` modes on multiline input.

Practical Examples

Key Points

- Email validation:

```
^[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$
```

- Phone number:

```
^\\d{10}$
```

- Date pattern:

```
\\d{4}-\\d{2}-\\d{2}
```

Code Snippet

```
Pattern email = Pattern.compile("^[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$");  
System.out.println(email.matcher("rohan@learnato.com").matches()); // true
```

 **Exercise:** Exercise_09_RealWorldPatterns.java

Write regex for email, mobile, and date validation.

Key Takeaways

- Regex enables **powerful text search, match, and validation**.
- Use `Pattern` to compile once, `Matcher` to reuse.
- Understand difference between `find()` and `matches()`.
- Use **groups** for extracting structured data.
- Flags make patterns flexible for real-world inputs.

Module – Java Date-Time API (LocalDate, LocalTime, LocalDateTime, Instant)

Why a New Date-Time API?

Key Points

- The old classes (`java.util.Date`, `Calendar`) were mutable and confusing.
- Introduced in **Java 8** under `java.time` package.
- Immutable, thread-safe, and follows ISO-8601 standard.
- Divided into **human time** (`LocalDate`, `LocalTime`, `LocalDateTime`) and **machine time** (`Instant`).

Code Snippet

```
import java.time.*;  
LocalDate today = LocalDate.now();  
System.out.println(today); // 2025-10-21
```

 **Exercise:** `Exercise_01_NewAPIIntro.java`

Compare outputs of `new Date()` vs `LocalDate.now()` and note the difference in readability.

LocalDate: Working with Dates

Key Points

- Represents **only the date** (no time or timezone).
- Immutable and created using `LocalDate.now()`, `LocalDate.of()`, or `LocalDate.parse()`.
- Common methods:
 - `plusDays()`, `minusWeeks()`, `getDayOfWeek()`, `isLeapYear()`.

Code Snippet

```
LocalDate date = LocalDate.of(2025, 10, 21);
System.out.println(date.getDayOfWeek()); // TUESDAY
System.out.println(date.plusDays(10));    // 2025-10-31
System.out.println(date.isLeapYear());    // false
```

 **Exercise:** `Exercise_02_LocalDateOps.java`

Perform operations like add/subtract days, months, years, and print results.

LocalTime: Representing Time of Day

Key Points

- Represents **time without date or zone** (hours, minutes, seconds, nanos).
- Use `LocalTime.now()`, `of()`, or `parse()`.
- Common methods:
 - `plusHours()`, `minusMinutes()`, `isBefore()`, `isAfter()`.

Code Snippet

```
LocalTime now = LocalTime.now();  
System.out.println(now); // e.g., 14:35:27.123  
System.out.println(now.plusHours(2)); // 16:35:27.123
```

 **Exercise:** `Exercise_03_LocalTimeOps.java`

Calculate working shift end times and compare times for morning/evening slots.

LocalDateTime: Combining Date and Time

Key Points

- Combines both date and time (still no timezone).
- Immutable, created using `now()`, `of()`, or `parse()`.
- Useful for local timestamp operations.

Code Snippet

```
LocalDateTime meeting = LocalDateTime.of(2025, 10, 21, 10, 30);
System.out.println(meeting.plusHours(5));    // 2025-10-21T15:30
System.out.println(meeting.toLocalDate());   // 2025-10-21
System.out.println(meeting.toLocalTime());   // 10:30
```

 **Exercise:** Exercise_04_LocalDateTime.java

Create a meeting schedule and shift times by adding durations.

Instant: Machine Time (UTC-based)

Key Points

- Represents a point in time in **UTC** (machine-readable).
- Measured as seconds/nanoseconds since the **epoch** (1970-01-01T00:00Z).
- Useful for timestamps, logging, and persistence.

Code Snippet

```
Instant now = Instant.now();  
System.out.println(now); // 2025-10-21T09:10:30.456Z  
System.out.println(now.getEpochSecond()); // e.g., 1768991430
```

 **Exercise:** Exercise_05_InstantDemo.java

Print current epoch seconds and convert Instant to LocalDateTime using system zone.

Converting Between Types

Key Points

- Use `ZoneId` to convert between `Instant` and `LocalDateTime`.
- Time zones provide **context** for machine timestamps.

Code Snippet

```
ZoneId zone = ZoneId.systemDefault();
Instant instant = Instant.now();
LocalDateTime local = LocalDateTime.ofInstant(instant, zone);
System.out.println(local);
```

 **Exercise:** `Exercise_06_ZoneConversion.java`

Convert `Instant` to local time for multiple time zones (Asia/Kolkata, UTC, US/Eastern).

Parsing & Formatting Dates

Key Points

- Use `DateTimeFormatter` to parse or format dates.
- Common patterns:
 - `"yyyy-MM-dd", "dd/MM/yyyy", "yyyy-MM-dd HH:mm:ss".`

Code Snippet

```
import java.time.format.*;
LocalDate date = LocalDate.parse("21/10/2025",
DateTimeFormatter.ofPattern("dd/MM/yyyy"));
String formatted = date.format(DateTimeFormatter.ofPattern("EEEE, MMM dd
yyyy"));
System.out.println(formatted); // Tuesday, Oct 21 2025
```

 **Exercise:** `Exercise_07_ParseFormat.java`

Parse and reformat dates in multiple patterns and locales.

Period, Duration & Between

Key Points

- `Period` → difference in **days, months, years**.
- `Duration` → difference in **hours, minutes, seconds**.
- `ChronoUnit.between()` → direct difference in chosen units.

Code Snippet

```
LocalDate start = LocalDate.of(2025, 1, 1);
LocalDate end = LocalDate.of(2025, 10, 21);
Period p = Period.between(start, end);
System.out.println(p); // P9M20D

LocalTime t1 = LocalTime.of(9, 0);
LocalTime t2 = LocalTime.of(17, 30);
System.out.println(Duration.between(t1, t2)); // PT8H30M
```

 **Exercise:** `Exercise_08_PeriodDuration.java`

Find duration between two times and period between two dates.

Key Takeaways

- `LocalDate`, `LocalTime`, and `LocalDateTime` → for human-readable values.
- `Instant` → for precise timestamps (machine time).
- Immutability ensures thread safety.
- Use `DateTimeFormatter` for custom parsing and display.
- Use `Period`/`Duration` for time differences and calculations.

Module – Parsing and Formatting using `DateTimeFormatter`

Why `DateTimeFormatter`?

Key Points

- Part of `java.time.format` package (Java 8+).
- Replaces old `SimpleDateFormat` (thread-unsafe).
- Used for both **parsing** (text → date) and **formatting** (date → text).
- Thread-safe, immutable, and supports **locale-aware** formats.

Code Snippet

```
import java.time.*;
import java.time.format.*;

LocalDate today = LocalDate.now();
String formatted = today.format(DateTimeFormatter.ISO_DATE);
System.out.println(formatted); // 2025-10-21
```

 **Exercise:** `Exercise_01_BasicFormatter.java`
Format current date using built-in ISO formatters.

Built-in Formatters

Key Points

- Java provides several ready-to-use formatters:

- `DateTimeFormatter.ISO_LOCAL_DATE` → 2025-10-21
- `DateTimeFormatter.ISO_LOCAL_DATE_TIME` → 2025-10-21T10:30:15
- `DateTimeFormatter.BASIC_ISO_DATE` → 20251021
- `DateTimeFormatter.ISO_OFFSET_DATE_TIME` → includes time zone offset.
 - Useful for interoperability with APIs and databases.

Code Snippet

```
LocalDateTime now = LocalDateTime.now();  
System.out.println(now.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));
```

 **Exercise:** `Exercise_02_BuiltInFormatters.java`

Print current date and time using all ISO variations.

Custom Format Patterns

Key Points

- Create your own pattern using `DateTimeFormatter.ofPattern()`.
- Pattern letters:
 - `y` → Year, `M` → Month, `d` → Day
 - `H` → Hour (0–23), `h` → Hour (1–12), `m` → Minute, `s` → Second
 - `E` → Day name, `a` → AM/PM

Pattern	Example Output
"dd/MM/yyyy"	21/10/2025
"EEEE, MMM dd yyyy"	Tuesday, Oct 21 2025
"hh:mm a"	10:30 AM

Code Snippet

```
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("EEEE, dd MMM yyyy");  
System.out.println(LocalDate.now().format(fmt));
```

 **Exercise:** `Exercise_03_CustomPatterns.java`
Create and print date/time in 3 different custom formats.

Parsing Strings into Dates

Key Points

- Use `LocalDate.parse()` or `LocalDateTime.parse()` to convert text to date/time.
- Use matching `DateTimeFormatter` for custom patterns.

Code Snippet

```
String text = "21/10/2025";  
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");  
LocalDate date = LocalDate.parse(text, fmt);  
System.out.println(date); // 2025-10-21
```

 **Exercise:** `Exercise_04_ParseCustomDate.java`

Parse strings like `05-11-2025` and `2025/11/05` using correct patterns.

Parsing and Formatting LocalDateTime

Key Points

- Works the same for date-time combinations.
- Must match both date and time parts in the input string.

Code Snippet

```
String text = "21-10-2025 14:45";
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
LocalDateTime dt = LocalDateTime.parse(text, fmt);
System.out.println(dt.format(DateTimeFormatter.ofPattern("EEE, h:mm a")));
// Tue, 2:45 PM
```

 **Exercise:** Exercise_05_ParseLocalDateTime.java

Parse date-time strings in different formats and reformat them with AM/PM.

Locale-Based Formatting

Key Points

- Localize date/time display automatically using `withLocale(Locale.XYZ)`.
- Common examples:
 - `Locale.US, Locale.FRANCE, Locale.JAPAN, Locale.of("hi", "IN")`
 - Ensures region-specific month/day names and order.

Code Snippet

```
import java.util.*;  
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("EEEE, dd MMMM yyyy",  
Locale.FRENCH);  
System.out.println(LocalDate.now().format(fmt)); // mardi, 21 octobre 2025
```

 **Exercise:** `Exercise_06_LocaleFormats.java`
Format today's date in US, France, Japan, and India locales.

Predefined Localized Formatters

Key Points

- Use localized styles for common date/time representations:
 - `FormatStyle.FULL`, `FormatStyle.LONG`, `FormatStyle.MEDIUM`, `FormatStyle.SHORT`.
 - Use `DateTimeFormatter.ofLocalizedDate()` or `.ofLocalizedDateTime()`.

Code Snippet

```
import java.time.format.FormatStyle;

LocalDate today = LocalDate.now();
System.out.println(today.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL)));
System.out.println(today.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)));
```

 **Exercise:** `Exercise_07_LocalizedFormats.java`

Print current date and time using all four `FormatStyle` levels.

Combining Parsing and Formatting

Key Points

- Read a date in one format → convert to another using formatters.
- Useful in data migration or UI display.

Code Snippet

```
String input = "21-10-2025";
DateTimeFormatter inputFmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
DateTimeFormatter outputFmt = DateTimeFormatter.ofPattern("EEEE, MMM dd
YYYY");
LocalDate date = LocalDate.parse(input, inputFmt);
System.out.println(date.format(outputFmt)); // Tuesday, Oct 21 2025
```

 **Exercise:** Exercise_08_ConvertFormats.java

Convert multiple input date strings into a uniform formatted output.

Handling Exceptions & Edge Cases

Key Points

- `DateTimeParseException` occurs when input doesn't match pattern.
- Always wrap parsing in try-catch for user inputs.
- Handle locale or missing fields (e.g., missing year).

Code Snippet

```
try {
    LocalDate.parse("2025.10.21", DateTimeFormatter.ofPattern("dd-MM-
    yyyy"));
} catch (Exception e) {
    System.out.println("Invalid format: " + e.getMessage());
}
```

 **Exercise:** `Exercise_09_ParseExceptionHandling.java`
Validate various date inputs and handle invalid formats gracefully.

Key Takeaways

- `DateTimeFormatter` = modern, safe replacement for `SimpleDateFormat`.
- Use **custom patterns** for flexibility, **localized styles** for i18n.
- Parsing ↔ Formatting uses consistent patterns.
- Always match pattern exactly to input format.
- Wrap parsing in try-catch for robust user input handling.

Module – Time Zones and Offsets (**ZoneId**, **ZonedDateTime**)

Why Time Zones Matter

Key Points

- Time zones determine how a global **Instant** is represented locally.
- Local time depends on region and offset from UTC (e.g., +05:30 for India).
- Java’s modern Date-Time API provides **zone-aware** classes like `ZoneId` and `ZonedDateTime`.
- Old `Date` and `Calendar` classes handled this inconsistently and were error-prone.

Code Snippet

```
import java.time.*;  
System.out.println(ZoneId.systemDefault()); // e.g., Asia/Kolkata
```

 **Exercise:** `Exercise_01_WhyTimeZones.java`

Print system default zone and check differences for a few common regions.

Understanding `ZoneId`

Key Points

- Represents a specific **region-based time zone**, e.g., "Asia/Kolkata", "America/New_York".
- `ZoneId.of("region/city")` gives full zone rules (including Daylight Savings).
- Use `ZoneOffset(+05:30)` for fixed offset zones.

Code Snippet

```
ZoneId india = ZoneId.of("Asia/Kolkata");
ZoneId ny = ZoneId.of("America/New_York");
System.out.println(india);
System.out.println(ny);
```

 **Exercise:** `Exercise_02_ZoneIdBasics.java`

List and print various zone IDs from `ZoneId.getAvailableZoneIds()`.

Working with `ZonedDateTime`

Key Points

- `ZonedDateTime = LocalDateTime + ZoneId`.
- Represents a specific date-time **with full time-zone context**.
- Created using `ZonedDateTime.now(zone)` or `of(LocalDateTime, ZoneId)`.

Code Snippet

```
LocalDateTime local = LocalDateTime.of(2025, 10, 21, 10, 30);
ZoneId india = ZoneId.of("Asia/Kolkata");
ZonedDateTime meeting = ZonedDateTime.of(local, india);
System.out.println(meeting); // 2025-10-21T10:30+05:30[Asia/Kolkata]
```

 **Exercise:** `Exercise_03_CreateZonedDateTime.java`

Create `ZonedDateTime` for same `LocalDateTime` in 3 regions — India, London, Tokyo.

Converting Between Time Zones

Key Points

- Use `withZoneSameInstant()` to convert exact instant to another zone.
- Use `withZoneSameLocal()` to keep same clock time, but change zone context.
- This is essential for scheduling across time zones.

Code Snippet

```
ZonedDateTime indiaTime = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));
ZonedDateTime nyTime =
indiaTime.withZoneSameInstant(ZoneId.of("America/New_York"));
System.out.println("India: " + indiaTime);
System.out.println("New York: " + nyTime);
```

 **Exercise:** `Exercise_04_TimeZoneConversion.java`

Convert between at least three zones (Asia, Europe, America) and print differences.

Understanding `ZoneOffset`

Key Points

- `ZoneOffset` represents a **fixed** difference from UTC (e.g., `+05:30`).
- Unlike `ZoneId`, it doesn't handle daylight saving changes.
- Used internally by `ZonedDateTime` for offset calculations.

Code Snippet

```
ZoneOffset offset = ZoneOffset.of("+05:30");
OffsetDateTime odt = OffsetDateTime.now(offset);
System.out.println(odt); // 2025-10-21T10:30+05:30
```

 **Exercise:** `Exercise_05_ZoneOffset.java`

Print `OffsetDateTime` for various offsets like `+00:00`, `+09:00`, `-04:00`.

From `Instant` to Zoned Time

Key Points

- `Instant` represents a timestamp in UTC.
- Convert to local time using `ZonedDateTime.ofInstant()`.
- Every `Instant` can have multiple local representations depending on zone.

Code Snippet

```
Instant now = Instant.now();
ZoneId tokyo = ZoneId.of("Asia/Tokyo");
ZonedDateTime tokyoTime = ZonedDateTime.ofInstant(now, tokyo);
System.out.println("UTC Instant: " + now);
System.out.println("Tokyo Time: " + tokyoTime);
```

 **Exercise:** `Exercise_06_InstantToZone.java`
Convert one `Instant` into times for 5 major cities.

Daylight Saving Time (DST) Awareness

Key Points

- Some zones adjust clocks (e.g., US, Europe).
- `ZonedDateTime` automatically handles DST changes based on rules.
- No manual adjustment required.

Code Snippet

```
ZoneId ny = ZoneId.of("America/New_York");
ZonedDateTime beforeDST = ZonedDateTime.of(2025, 3, 9, 1, 30, 0, 0, ny);
ZonedDateTime afterDST = beforeDST.plusHours(2);
System.out.println(beforeDST); // Before DST shift
System.out.println(afterDST);  // After DST shift (clock jumps)
```

 **Exercise:** `Exercise_07_DSTBehavior.java`

Print time transitions around DST boundaries and note clock adjustments.

Formatting ZonedDateTime

Key Points

- Use `DateTimeFormatter` to display time with zone and offset info.
- Patterns:
 - "yyyy-MM-dd HH:mm z" → zone abbreviation.
 - "yyyy-MM-dd HH:mm O" → localized offset.
 - "yyyy-MM-dd HH:mm VV" → full zone name.

Code Snippet

```
import java.time.format.*;

ZonedDateTime zdt = ZonedDateTime.now();
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm z (VV)");
System.out.println(zdt.format(fmt));
// e.g., 2025-10-21 10:30 IST (Asia/Kolkata)
```

 **Exercise:** Exercise_08_FormatZones.java

Print `ZonedDateTime` in multiple formats showing zone and offset.

Comparing ZonedDateTime

Key Points

- Use `isBefore()`, `isAfter()`, `equals()` to compare instants across zones.
- Even if local times differ, `ZonedDateTime` normalizes via `Instant`.

Code Snippet

```
ZonedDateTime india = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));
ZonedDateTime london =
india.withZoneSameInstant(ZoneId.of("Europe/London"));
System.out.println(india.isEqual(london)); // true (same instant)
```

 **Exercise:** `Exercise_09_CompareZones.java`

Compare equivalent `ZonedDateTime`s across multiple regions.

Key Takeaways

- `ZoneId` → identifies geographic region with time rules.
- `ZonedDateTime` → date + time + zone info (DST-aware).
- Use `ZoneOffset` for fixed UTC offsets (no DST).
- Use `withZoneSameInstant()` for true conversions.
- Always store timestamps in UTC (`Instant`) and display in user's zone.

Module – Period, Duration, and Daylight-Saving Adjustments

Why We Need Period and Duration

Key Points

- `Period` and `Duration` measure **differences between two temporal values**.
- `Period` → human time (years, months, days).
- `Duration` → machine time (hours, minutes, seconds, nanoseconds).
- Both are **immutable** and part of `java.time` package.

Code Snippet

```
import java.time.*;  
LocalDate start = LocalDate.of(2025, 1, 1);  
LocalDate end = LocalDate.of(2025, 10, 21);  
Period diff = Period.between(start, end);  
System.out.println(diff); // P9M20D
```

 **Exercise:** `Exercise_01_PeriodVsDuration.java`

Compute difference between two dates and two times, and print their `Period` and `Duration`.

Working with `Period`

Key Points

- Used for **calendar-based differences** — ignores hours/minutes.
- Created using:
 - `Period.ofYears(1)`, `Period.ofMonths(6)`, or `Period.between(date1, date2)`.
 - Supports addition and subtraction using `plus()` and `minus()`.

Code Snippet

```
Period p1 = Period.ofMonths(3);
Period p2 = Period.ofWeeks(2);
LocalDate today = LocalDate.now();
System.out.println(today.plus(p1)); // +3 months
System.out.println(today.minus(p2)); // -2 weeks
```

 **Exercise:** `Exercise_02_PeriodOps.java`

Experiment by adding/subtracting different `Periods` to end-of-month dates.

Period Details and Components

Key Points

- You can retrieve individual components:
 - `getYears()`, `getMonths()`, `getDays()`.
 - To get total days between dates, use `ChronoUnit.DAYS.between()`.

Code Snippet

```
Period diff = Period.between(LocalDate.of(2025, 1, 1), LocalDate.of(2025, 10, 21));
System.out.printf("%d years, %d months, %d days%n",
    diff.getYears(), diff.getMonths(), diff.getDays());
System.out.println("Total days: " + ChronoUnit.DAYS.between(
    LocalDate.of(2025, 1, 1), LocalDate.of(2025, 10, 21)));
```

 **Exercise:** `Exercise_03_PeriodDetails.java`

Calculate years, months, and days between user-input birthdays.

Working with `Duration`

Key Points

- Represents a **fixed time-based amount** (seconds/nanos).
- Used for time calculations like stopwatch, task timers, etc.
- Can be created using:
 - `Duration.ofHours(2)`, `Duration.ofMinutes(30)`, **or** `Duration.between(time1, time2)`.

Code Snippet

```
LocalTime start = LocalTime.of(9, 0);
LocalTime end = LocalTime.of(17, 30);
Duration d = Duration.between(start, end);
System.out.println(d); // PT8H30M
System.out.println(d.toHours()); // 8
System.out.println(d.toMinutes()); // 510
```

 **Exercise:** `Exercise_04_DurationOps.java`
Calculate duration between login and logout times.

Comparing Period and Duration

Key Points

Feature	Period	Duration
Based on	Date	Time
Unit	Years, months, days	Hours, minutes, seconds
Variable length months	✓	✗
Suitable for	Calendar calculations	Stopwatch/timers
Example	Days till next birthday	Time taken to execute a job

 **Exercise:** `Exercise_05_PeriodDurationCompare.java`

Compute both `Period` and `Duration` between two `LocalDateTime` objects and compare differences.

Using ChronoUnit for Quick Differences

Key Points

- ChronoUnit provides direct `between()` for any units — simpler than Period/Duration for quick calculations.
- Supports DAYS, MONTHS, YEARS, HOURS, MINUTES, etc.

Code Snippet

```
import java.time.temporal.ChronoUnit;
LocalDate start = LocalDate.of(2025, 1, 1);
LocalDate end = LocalDate.of(2025, 10, 21);
System.out.println(ChronoUnit.DAYS.between(start, end)); // 293
System.out.println(ChronoUnit.MONTHS.between(start, end)); // 9
```

 **Exercise:** Exercise_06_ChronoUnit.java

Measure intervals in various units (hours, days, months).

Daylight-Saving Time (DST) Challenge

Key Points

- DST causes clocks to **jump forward or backward**, affecting Duration but not Period.
- ZonedDateTime automatically accounts for DST changes.
- Always use **ZonedDateTime** + Duration.between() for accurate elapsed time in zones with DST.

Code Snippet

```
ZoneId ny = ZoneId.of("America/New_York");
ZonedDateTime beforeDST = ZonedDateTime.of(2025, 3, 9, 1, 30, 0, 0, ny);
ZonedDateTime afterDST = beforeDST.plusHours(2);
Duration diff = Duration.between(beforeDST, afterDST);
System.out.println(diff.toHours()); // 1 hour due to DST skip
```

 **Exercise:** Exercise_07_DSTDuration.java

Compute duration across DST boundaries and explain difference between local vs real elapsed time.

Adjusting for DST in Calculations

Key Points

- Use `ZonedDateTime` to avoid manual correction for DST.
- To compare across DST changes:
 - Use `withZoneSameInstant()` for true global comparison.
 - Use `Instant` for machine-based timestamps.

Code Snippet

```
ZonedDateTime start = ZonedDateTime.of(2025, 10, 26, 1, 0, 0, 0,
ZoneId.of("Europe/London"));
ZonedDateTime end = start.plusHours(2);
System.out.println(Duration.between(start, end)); // PT3H due to clock
rollback
```

 **Exercise:** `Exercise_08_DSTAdjustments.java`

Compare durations before and after DST transitions in multiple regions.

Combining Period, Duration, and Time Zones

Key Points

- Period + Duration = complete time difference representation.
- Use Period for date-level logic (deadlines, billing cycles).
- Use Duration for precise time-level tracking (elapsed time, logs).
- Handle time zones carefully to account for daylight shifts.

Code Snippet

```
ZonedDateTime start = ZonedDateTime.of(2025, 1, 1, 8, 0, 0, 0,
ZoneId.of("Asia/Kolkata"));
ZonedDateTime end = ZonedDateTime.now();
Period dateGap = Period.between(start.toLocalDate(), end.toLocalDate());
Duration timeGap = Duration.between(start.toLocalTime(),
end.toLocalTime());
System.out.println("Elapsed: " + dateGap + " and " + timeGap);
```

 **Exercise:** Exercise_09_PeriodDurationTogether.java

Compute both calendar and precise duration gaps between events in different zones.

Key Takeaways

- `Period` → for date differences (years, months, days).
- `Duration` → for time differences (hours, minutes, seconds).
- `ChronoUnit` → for direct unit-based calculations.
- `ZonedDateTime` handles DST automatically.
- Use both together for accurate real-world date/time analytics.